

SPM: Concurrency in C++

Academic year 2025-2026

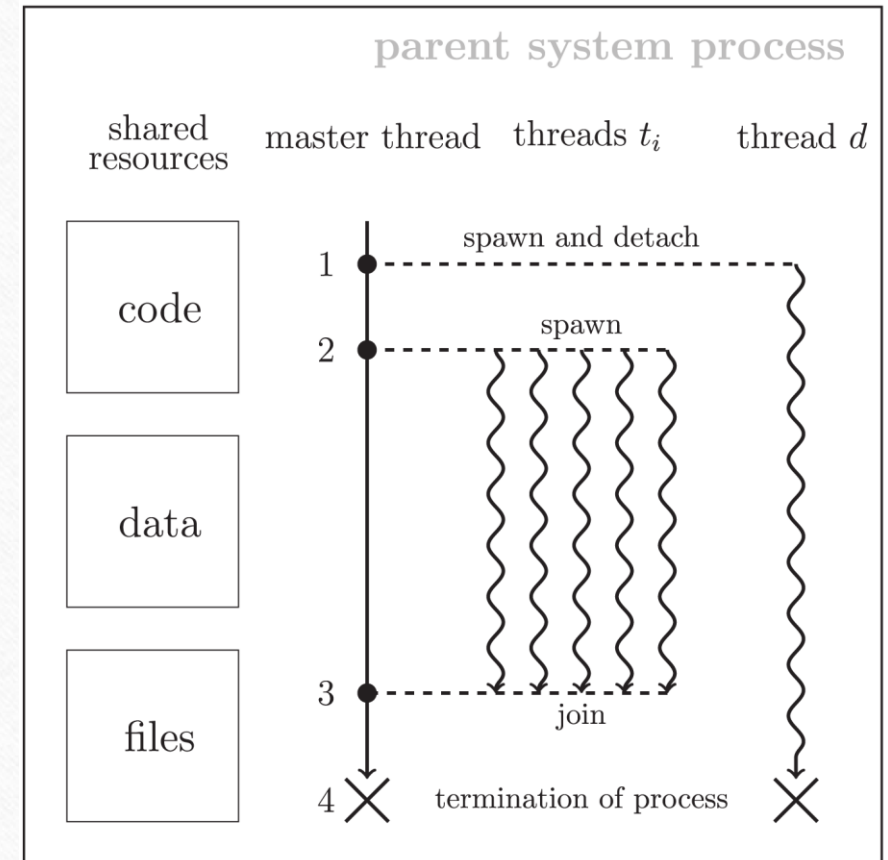
Prof. Massimo Torquati
massimo.torquati@unipi.it

Outline

- Spawning C++ threads
- Future, Promise and Async
- Task objects
- Mutexes and Condition Variables
- Classic Concurrency Problems

Spawning and joining C++ threads

- The master thread can spawn threads, and each thread can spawn threads as well
- The number of spawned threads should be roughly the amount of cores (i.e., pay attention to **oversubscription**)
- Threads share process resources
 - each thread has a separate stack
- A thread can be **joined** or **detached** once
- A detached thread cannot be joined
- Joined or detached threads cannot be reused
- All threads must be joined or detached within the scope of their declaration (i.e., before the `std::thread` object will be destroyed)



Spawning and joining C++ threads

- We need to store the thread handles explicitly to be able to access them during the join phase
- In the code snippet, we use a `std::vector` container and the method `emplace_back`
- Alternatively, we could have **moved** the thread object implicitly using the `vector` member function `push_back`

```
threads.push_back(std::thread(say_hello, id));
```

- The type `std::thread` is **move-only** (i.e., not copyable)
 - Copy constructor and copy assignment operator deleted (i.e., `=delete`)
- How to compile:

```
g++ -O3 -std=c++20 spawn-and-join.cpp -o spawn -pthread
```

```
spawn-and-join.cpp > ...
1  #include <cstdlib> // uint64_t
2  #include <iostream> // std::cout std::endl
3  #include <vector>   // std::vector
4  #include <thread>   // std::thread
5
6  // this function will be called by the threads (should be void)
7  void say_hello(uint64_t id) {
8      std::cout << "Hello from thread: " << id << std::endl;
9  }
10
11 // this runs in the master thread
12 int main(int argc, char * argv[]) {
13
14     const uint64_t num_threads{(argc==2)?std::stoul(argv[1]):10};
15     std::vector<std::thread> threads;
16
17     // for all threads
18     for (uint64_t id = 0; id < num_threads; id++)
19         // emplace the thread object in vector threads
20         // using argument forwarding, this avoids unnecessary
21         // move operations to the vector after thread creation
22         threads.emplace_back(
23             // call say_hello with argument id
24             say_hello, id
25         );
26
27     // join each thread at the end
28     for (auto& thread: threads)
29         thread.join();
30 }
```


Passing parameters to threads

- Let's consider a simple scalar function that iteratively computes the n-th Fibonacci's number

$$a_n = a_{n-1} + a_{n-2} \quad \text{with initial conditions} \quad a_0 = 0, a_1 = 1$$

- In the code snippet we show two different ways to pass arguments to the thread functions; the second argument is **passed by reference** to store the result
- Potential pitfalls:
 - The memory passed via a reference (or pointer) has to be persistent during the execution of threads
 - The memory used for variables or objects defined within the loop body where threads are spawned is destroyed when leaving the scope (the destructor is called)

```
fibonacci.cpp > ...
3  #include <vector>    // std::vector
4  #include <thread>    // std::thread
5
6  template <typename value_t>
7  void fibo(value_t n,
8            value_t & result) { // <- here we pass the reference
9      value_t a_0 = 0, a_1 = 1;
10     for (uint64_t index = 0; index < n; index++) {
11         const value_t tmp = a_0; a_0 = a_1; a_1 += tmp;
12     }
13     result = a_0;
14 }
15
16 // this runs in the master thread
17 int main(int argc, char * argv[]) {
18     const uint64_t num_threads = 32;
19     std::vector<std::thread> threads;
20
21     // allocate num_threads many result values
22     std::vector<uint64_t> results(num_threads, 0);
23
24     for (uint64_t id = 0; id < num_threads; id++)
25         threads.emplace_back(
26             // specify template parameters and arguments
27             // fibo<uint64_t>, id, std::ref(results[id])
28
29             // using lambda and automatic type deduction
30             // pay attention to id, must be passed by value!
31             [id, &results]() { fibo(id, results[id]); }
32         );
33     // join the threads
34     for (auto& thread: threads)
35         thread.join();
36     // print the result
37     for (const auto& result: results)
38         std::cout << result << std::endl;
39 }
```


Mutex variables

- A mutex is a synchronization mechanism that guarantees **mutual exclusion** execution of a critical section. Its usage restricts the execution of a critical section to a single thread at a time.
- A thread locking a mutex prevents other threads from acquiring the mutex. The other threads **wait** for its release (**passive waiting**)

```
#include <mutex>
std::mutex mutex;

// to be called by threads
void some_function(...) {

    mutex.lock()
    // this region is only processed by one thread at a time
    mutex.unlock();

    // this region is processed in parallel
}
```

```
#include <mutex>
std::mutex mutex;

// to be called by threads
void some_function(...) {

    {
        // here we acquire the lock
        std::lock_guard<std::mutex> lock_guard(mutex);
        // this region is locked by the mutex
    } // <- here we release the lock

    // this region is processed in parallel
}
```

lock_guard vs. unique_lock vs. scoped_lock

- *std::lock_guard* is a RAII wrapper around *std::mutex* that acquires ownership of the mutex upon construction and releases it upon destruction
 - Once constructed, it cannot be explicitly released until it goes out of scope
 - It can only manage a single mutex
- *std::unique_lock* is more flexible and supports lock/unlock if needed
 - It can be moved to transfer ownership of the lock
 - Often used with condition variables
- *std::scoped_lock* manages multiple mutexes simultaneously, acquiring/releasing all of them simultaneously when entering/exiting the scope
 - It provides an unlock() method to explicitly release the locks before the end of the scope
 - Helps avoid deadlock when you need to acquire multiple locks together (using a lock ordering strategy)
 - Pay attention to the fact that it accepts 0 mutexes; it simply does nothing

Condition Variables (CV)

- A CV enables one or more threads **to wait (passively) for an event inside a critical section**
 - Conceptually, a CV is associated with an **event or condition**. When a thread has determined that the condition is satisfied, it can **notify** one or more of the threads waiting on the CV to wake them up
 - **Pay attention to spurious wakeups!** The condition must be checked in a loop (typically a while loop) or the **wait** with the predicate version should be used.

Workflow of the signaling thread

1. The signaling thread has to acquire a mutex using either `mutex.lock()` or through a scoped wrapper such as `std::lock_guard` or `std::unique_lock`.
2. While holding the lock, the shared state is modified.
3. The lock is released either explicitly with `mutex.unlock()`, or implicitly by leaving the scope of `std::lock_guard` or `std::unique_lock`.
4. The actual signaling of the condition variable `cv` is performed using `cv.notify_one()`, or `cv.notify_all()`

Note that 3 and 4 can be swapped! **Slower but usually safer!**

Workflow of the waiting thread(s)

1. Acquire a **`std::unique_lock`** using the same mutex as in the signaling phase. Note that `std::lock_guard` cannot be used here.
2. While being locked call either `cv.wait()`, `cv.wait_for()`, or `wait_until()`. The lock is released automatically (**and atomically**).
3. In case (i) `cv` is notified, (ii) the timeout of `cv.wait()` or `cv.wait_for()` expires, or (iii) a spurious wake-up occurs, then the thread is awakened, and the lock is reacquired. At this point, we must check whether the globally shared state (the condition) indicates proceeding or waiting (i.e., sleeping) again.

Condition Variables usage pattern

- Typical pattern for the signaler

```
{ // notify while holding the lock
  std::lock_guard<std::mutex> lock(mtx);
  ... // modify the predicate (wait-cond)
  cv.notify_one(); // may also use notify_all()
}
```

```
{ // unlock, then notify (better for performance)
  std::lock_guard<std::mutex> lock(mtx);
  ... // modify the predicate (wait-cond)
}
cv.notify_one(); // may also use notify_all()
```

- Typical pattern for the waiter

```
{ // wait in a while loop
  std::unique_lock<std::mutex> lock(mtx);
  ...
  while( !wait-cond ) { // re-check after every wakeup
    cv.wait(lock); //atomically unlock and wait, then re-lock
  }
  ...
}
```

```
{ // wait using a predicate in the wait method
  std::unique_lock<std::mutex> lock(mtx);
  ...
  cv.wait(lock, [&]()) { // return true when it is safe to go on
    return wait-cond; // if false, atomically unlock and wait
  }); // on wakeup re-lock and re-check the wait-cond
  ...
}
```


Producer-Consumer pattern

- Classical concurrency pattern involving two types of threads: `producers`, and `consumers`
- Producers: generate data items and add them to a shared data structure (usually a queue that can have bounded or unbounded capacity)
- Consumers: retrieve data items from the shared data structure (usually one at a time) and consume them doing some work
- Let's see an implementation in which the buffer is unbounded in size (implemented through a `std::deque` container) and the classical implementation with mutex and condition variables.
 - Code: `prod-cons.cpp` (unbounded queue version), `prod-cons-b.cpp` (bounded queue version)

Example: alarm clock

- In the code snippet a thread executes the student function, which checks, in a passive-wait loop, a global variable (*time_for_breakfast*)
- The master thread, sets the global variable after a while and notifies the student thread (line 38)
- Remember to use *wait* on a CV either in a while loop or, alternatively, using the *wait* method with a predicate

```
while(wait-condition) {  
    cv.wait(lock);  
}
```

```
cv.wait(lock, [&]() {  
    return !wait-condition;  
});
```

```
alarm_clock.cpp > ...  
4  #include <chrono>           // std::this_thread::sleep_for  
5  #include <condition_variable> // std::condition_variable  
6  // convenient time formats  
7  using namespace std::chrono_literals;  
8  
9  int main() {  
10     std::mutex mutex;  
11     std::condition_variable cv;  
12     bool time_for_breakfast = false; // globally shared state  
13     // to be called by thread  
14     auto student = [&] () -> void {  
15         { // this is the scope of the lock  
16             std::unique_lock<std::mutex> unique_lock(mutex);  
17  
18             // check the globally shared state  
19             while (!time_for_breakfast)  
20                 // lock is released during wait  
21                 cv.wait(unique_lock);  
22  
23             // alternative  
24             //cv.wait(unique_lock,  
25             //         [&]() { return time_for_breakfast; });  
26         } // lock is finally released  
27         std::cout << "Time to make coffee!" << std::endl;  
28     };  
29  
30     // create the waiting thread and wait for 2s  
31     std::thread my_thread(student);  
32     std::this_thread::sleep_for(2s);  
33     { // prepare the alarm clock  
34         std::lock_guard<std::mutex> lock_guard(mutex);  
35         time_for_breakfast = true;  
36     } // here the lock is released  
37     // ring the alarm clock  
38     cv.notify_one();  
39     // wait until breakfast is finished  
40     my_thread.join();  
41 }
```


Attention!

- This is an **incorrect** implementation of the alarm clock example.
- The *sleep_for* calls at lines 20 and 31 have been inserted purposely to make it stall
- What's the problem? Why does it stall?
 - **Race condition**

```
alarm_clock_wrong.cpp > ...
4  #include <chrono>
5  #include <condition_variable>
6  // convenient time formats
7  using namespace std::chrono_literals;
8
9  int main() {
10     std::mutex mutex;
11     std::condition_variable cv;
12     bool time_for_breakfast = false; // globally shared state
13     // to be called by thread
14     auto student = [&] ( ) -> void {
15         { // this is the scope of the lock
16             std::unique_lock<std::mutex> unique_lock(mutex);
17
18             // check the globally shared state
19             while (!time_for_breakfast) {
20                 std::this_thread::sleep_for(1s);
21                 // lock is released during wait
22                 cv.wait(unique_lock);
23             }
24
25             }
26             std::cout << "Time to make coffee!" << std::endl;
27         };
28
29         // create the waiting thread and wait for 2s
30         std::thread my_thread(student);
31         std::this_thread::sleep_for(0.5s);
32         time_for_breakfast = true;
33         cv.notify_one();
34         // wait until breakfast is finished
35         my_thread.join();
36     }
37 }
```


One-shot notifications using futures

- CVs are helpful in case many threads perform non-trivial, or repetitive synchronization patterns
- Futures and promises can be used for the so-called *one-shot synchronization* scenario
- Synchronization between the issuing thread and the waiting threads is achieved since the access to a future's values via *f.get()* blocks until the fulfilling of the corresponding promise with *p.set_value()*
- The so-called *shared futures* can be used to **broadcast** a specific value to more than one thread

```
one_shot_alarm_clock.cpp > ...
1 > #include <iostream>           // std::cout...
5 // convenient time formats
6 using namespace std::chrono_literals;
7
8 int main() {
9     // create pair (future, promise)
10    std::promise<void> promise;
11    auto shared_future = promise.get_future().share();
12
13    // to be called by thread
14    auto students = [&] (int myid) -> void {
15        // blocks until fulfilling promise
16        shared_future.get();
17        std::cout << "[" << myid <<
18            "]" Time to make coffee!" << std::endl;
19    };
20
21    // create the waiting thread and wait for 2s
22    std::thread my_thread0(students, 0);
23    std::thread my_thread1(students, 1);
24    std::this_thread::sleep_for(2s);
25    promise.set_value();
26
27    // wait until breakfast is finished
28    my_thread0.join();
29    my_thread1.join();
30 }
```


Multiple-Reader Single-Writer pattern

- Classical concurrency pattern involving two types of threads: **readers** and **writers**
 - A reader may access the data concurrently with other readers
 - A writer needs exclusive access to the shared data
 - How does this differ from the **Producer-Consumer** pattern?
- Using a plain `std::mutex` to protect the accesses to the critical section **prevents** the concurrent access of multiple readers
- To implement the pattern, the C++17 standard provides `std::shared_mutex` and `std::shared_timed_mutex`
- No guarantees about fairness.
 - A continuous stream of readers can potentially starve a writer
 - If priority or fairness between readers and writers is required to avoid potential **starvation**, it must be explicitly implemented by the programmer

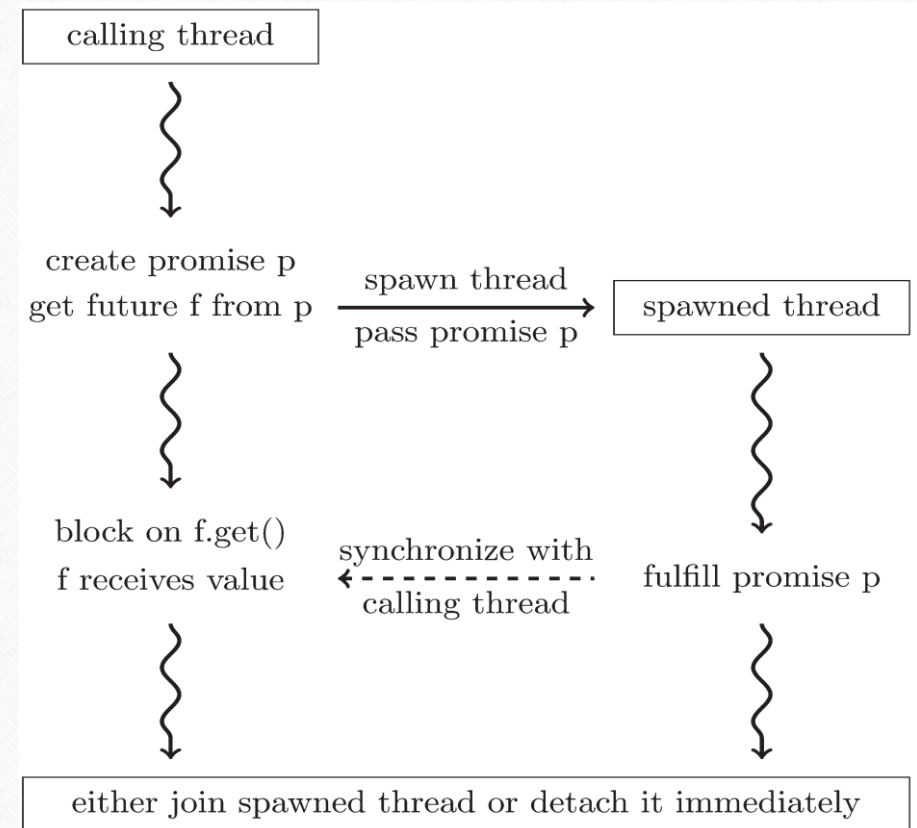
```
reader-writer.cpp > ...
1 #include <mutex>
2 #include <shared_mutex>
3
4 int main() {
5     int shared_counter = 0;
6     std::shared_mutex smutex;
7
8     auto reader = [&](int count, int id) {
9         for(int i=0; i<count; ++i) {
10             std::shared_lock<std::shared_mutex> lock(smutex);
11             std::printf("Reader%d has read %d\n", id, shared_counter);
12         }
13     };
14     auto writer = [&](int count, int id) {
15         for(int i=0; i<count; ++i) {
16             std::unique_lock<std::shared_mutex> lock(smutex);
17             ++shared_counter;
18             std::printf("Writer%d has written %d\n", id, shared_counter);
19         }
20     };
21
22     // Create multiple reader threads
23     std::vector<std::thread> readers;
24     for (int i = 0; i < 5; ++i) {
25         readers.emplace_back(reader, 4, i);
26     }
27
28     // Create multiple writer threads
29     std::vector<std::thread> writers;
30     for (int i = 0; i < 3; ++i) {
31         writers.emplace_back(writer, 3, i);
32     }
33
34     // Wait for all threads to finish.
35     for (auto& reader : readers) reader.join();
36     for (auto& writer : writers) writer.join();
37
38     return 0;
39 }
```


Promises and futures

- C++ provides a mechanism for return-value-passing specifically designed to fit the characteristics of **asynchronous execution**
 - The so-called ***promises*** that are *fulfilled in the future*.
- A promise and a future implement a “**single-assignment channel**” (a one-shot communication mechanism) between two entities, usually two threads
- $s = (p, f)$ where p is a writable view of the state s (the *promise*), f (the *future*), is a readable view of the state s that can be accessed after being signaled (**only once!**) by the promise
- The causal dependency between the *promise* p and the *future* f implements a **synchronization mechanism** between two threads

Promises and futures

- The state $s = (p, f)$ represents the logical channel. It is defined by a promise `std::promise<T> p` for a specific data type T , which has associated the `std::future<T> f = p.get_future()`
 - `std::promise<T> p` represents the write-end of the communication
 - `std::future<T> f` represents the read-end of the communication
- p is passed as *rvalue reference* in the signature of the called function via `std::promise<T> && p;`
 - p has to be moved using `std::move()` from the master thread to the spawned thread since `std::promise` is **not copiable**
- The promise p is fulfilled within the body of the spawned thread by `p.set_value(some_value);`
- Finally, we can read the future f in the master thread using `f.get()`, **blocking** its execution until f is signaled by p



Fibonacci example revisited (with promises)

- In the code snippet, we show how to pass a single promise to the thread to get the result
 - When using a lambda to call the thread function, pay attention that the promise cannot be passed by reference, it must be moved
 - The lambda must be *mutable* to make the captured entities *non-const* attributes of the closure object

```
Fibopromise.cpp > ...
4  #include <thread>    // std::thread
5  #include <future>    // std::future
6
7  template <typename value_t>
8  void fibo(value_t n,
9           std::promise<value_t> && result) { // <- pass promise
10     value_t a_0 = 0, a_1 = 1;
11     for (uint64_t index = 0; index < n; index++) {
12         const value_t tmp = a_0; a_0 = a_1; a_1 += tmp;
13     }
14     result.set_value(a_0); // fulfill promise
15 }
16
17 // this runs in the master thread
18 int main(int argc, char * argv[]) {
19     const uint64_t num_threads = 32;
20     std::vector<std::thread> threads;
21     // allocate num_threads many result values
22     std::vector<std::future<uint64_t>> results;
23
24     for (uint64_t id = 0; id < num_threads; id++) {
25         // define a promise and store the associated future
26         std::promise<uint64_t> promise;
27         results.emplace_back(promise.get_future());
28         threads.emplace_back(
29             // specify template parameters and arguments
30             // fibo<uint64_t>, id, std::move(promise)
31             // using lambda and automatic type deduction
32             // mutable avoids p (and also id) to be const
33             [id, p{std::move(promise)}]() mutable {fibo(id, std::move(p));}
34         );
35     }
36     // read the futures resulting in synchronization of threads
37     // up to the point where promises are fulfilled
38     for (auto& result: results) std::cout << result.get() << std::endl;
39     // join the threads
40     for (auto& thread: threads) thread.join();
41 }
```


Creating task objects with *std::packaged_tasks*

- The “*future*” C++ header provides the *std::package_task* function that allows you to conveniently construct **task objects**
- A task object is a **callable object** with the associated corresponding *future* object handling the return value
 - A callable object is any entity that can be invoked using the function **operator ()**. It includes *std::function* wrapper, functors, lambda expression, *std::bind* expression. A *callable* includes callable objects, regular functions, and function pointers.
- Example:

Assume we want to create a task from this function

```
bool comp(float value, int64_t threshold) {  
    return value < threshold;  
}
```

Create and use the task in this way:

```
// create the task and assign future  
std::packaged_task<bool(float, int64_t)> task(comp);  
auto future = task.get_future();  
  
// call the task with arguments  
task(value, threshold); // WARNING: this is sequential!  
  
// access future object (either true or false)  
std::cout << future.get() << std::endl;
```


Fibonacci example revisited (with tasks)

- We can create a specific task that calls the fibo function using `std::packaged_task`

```
std::packaged_task<uint64_t(uint64_t)> task(fibo);
```

- Note: `std::packaged_task` is a move-only type

```
fibotasks0.cpp x
fibotasks0.cpp > ...
1  #include <iostream> // std::cout
2  #include <cstdint>  // uint64_t
3  #include <vector>   // std::vector
4  #include <thread>   // std::thread
5  #include <future>   // std::future
6
7
8  // traditional signature of fibo
9  uint64_t fibo(uint64_t n) {
10     uint64_t a_0 = 0, a_1 = 1;
11     for (uint64_t index = 0; index < n; index++) {
12         const uint64_t tmp = a_0; a_0 = a_1; a_1 += tmp;
13     }
14     return a_0;
15 }
16
17 // this runs in the master thread
18 int main(int argc, char * argv[]) {
19     const uint64_t num_threads = 32;
20     std::vector<std::thread> threads;
21     // allocate num_threads many result values
22     std::vector<std::future<uint64_t>> results;
23
24     // create tasks, store futures and spawn threads
25     for (uint64_t id = 0; id < num_threads; id++) {
26         std::packaged_task<uint64_t(uint64_t)> task(fibo);
27         results.emplace_back(task.get_future());
28         threads.emplace_back(std::move(task), id);
29     }
30     for (auto& result: results) std::cout << result.get() << std::endl;
31     for (auto& thread: threads) thread.join();
32 }
```


Creating generic task objects

- The problem with the described approach is that the function signature is hard-coded in the template parameter of the `std::packaged_task`
`std::packaged_task<bool(float, int64_t)> task(comp);`
- It would be better if all tasks exhibit the same signature, e.g., **`void task(void)`**, independently of the callable object to be invoked
- This can be achieved by using a task factory function template (*`make_task`*)
- Purpose:
 - Custom scheduling of tasks with a dedicated thread pool
 - Allows to decouple the creation of the task from its execution
 - More control over error handling and cancellation

```
make_task.cpp > ...
3  #include <future>
4  #include <functional>
5
6  template<
7      typename Func,      // <-- type of the func
8      typename ... Args, // <-- arguments type arg0, arg1, ...
9      typename Rtrn=typename std::result_of<Func(Args...)>::type
10     >                    // ^-- type of the return value func(args)
11     auto make_task(
12         Func && func,
13         Args && ...args) -> std::packaged_task<Rtrn(void)> {
14
15     // basically build an auxilliary function aux(void)
16     // without arguments returning func(arg0,arg1,...)
17     auto aux = std::bind(std::forward<Func>(func),
18                         std::forward<Args>(args)...);
19
20     // create a task wrapping the auxilliary function:
21     // task() executes aux(void) := func(arg0,arg1,...)
22     auto task = std::packaged_task<Rtrn(void)>(aux);
23
24     // the return value of aux(void) is assigned to a
25     // future object accessible via task.get_future()
26     return task;
27 }
28
29 bool comp(float value, int64_t threshold) {
30     return value < threshold;
31 }
32
33 int main(int argc, char * argv[]) {
34     auto task = make_task(comp, 10.3, 20);
35     auto future = task.get_future();
36     // spawn a thread and detach it
37     std::thread thread(std::move(task));
38     thread.detach();
39     std::cout << future.get() << "\n";
40     // cannot join the thread, it is detached!
41 }
```


Fibonacci example revisited (generic tasks)

- The readability of the code has enhanced (but `make_task`).
- A thread executes the task
- The result is embedded in a `std::future`

```
fibotasks.cpp > ...
5  #include <future>    // std::future
6  #include <functional> // std::bind
7
8  template<
9      typename Func,    // <-- type of the func
10     typename ... Args, // <-- arguments type arg0, arg1, ...
11     typename Rtrn=typename std::result_of<Func(Args...)>::type
12     >                  // ^-- type of the return value func(args)
13 auto make_task(
14     Func && func,
15 >     Args && ...args) -> std::packaged_task<Rtrn(void)> { ...
30
31 // traditional signature of fibo
32 > uint64_t fibo(uint64_t n) { ...
39
40 // this runs in the master thread
41 int main(int argc, char * argv[]) {
42     const uint64_t num_threads = 32;
43     std::vector<std::thread> threads;
44     // allocate num_threads many result values
45     std::vector<std::future<uint64_t>> results;
46
47     // create tasks, store futures and spawn threads
48     for (uint64_t id = 0; id < num_threads; id++) {
49         auto task = make_task(fibo, id);
50         results.emplace_back(task.get_future());
51         threads.emplace_back(std::move(task));
52     }
53     for (auto& result: results) std::cout << result.get() << std::endl;
54     for (auto& thread: threads) thread.join();
55 }
```


The *async*chronous way

- The “*future*” C++ header provides an out-of-the-box `std::async` function for creating task objects
 - Higher-level interface for launching tasks asynchronously
- It executes a task object asynchronously using either a separate **thread or the calling thread** and returns a future
- However, there are pitfalls to pay attention to:

```
auto future = std::async(fibo, id);
```

- A call to `std::async` **does not necessarily imply that a new thread is spawned**
 - The calling thread might execute the task without spawning a new thread!
 - Default behavior depends on the implementation; thus, remember to use `std::launch::async`

```
auto future = std::async(std::launch::async, fibo, id);
```

- The execution of the task could be **deferred forever** if the future is not accessed (i.e., `future.get()/.wait()`)
- The execution of distinct tasks could be serialized. If the destructor of the future is called just after the `async`.

Fibonacci example revisited (with async)

- The code snippet of the Fibonacci example is rewritten using the `std::async` construct
- Pay attention to code like the following one:

```
for (uint64_t id = 0; id < num_threads; id++) {  
    auto future = std::async(std::launch::async, fibo, id);  
} // <- here, the destructor of future is called
```

- The destructor of the future can block until the async task completes (any exceptions are lost)
- The future must be stored outside the loop body
- The use of `std::async` is not advisable for methods without a return value

```
fibosync.cpp > ...  
1  #include <iostream> // std::cout  
2  #include <cstdint>  // uint64_t  
3  #include <vector>   // std::vector  
4  #include <future>   // std::async  
5  
6  uint64_t fibo(uint64_t n) {  
7      uint64_t a_0 = 0, a_1 = 1;  
8      for (uint64_t index = 0; index < n; index++) {  
9          const uint64_t tmp = a_0; a_0 = a_1; a_1 += tmp;  
10     }  
11     return a_0;  
12 }  
13  
14 int main(int argc, char * argv[]) {  
15     const uint64_t num_threads = 32;  
16  
17     std::vector<std::future<uint64_t>> results;  
18     // for each thread  
19     for (uint64_t id = 0; id < num_threads; id++)  
20         // directly emplace the future  
21         results.emplace_back(  
22             std::async(std::launch::async, fibo, id)  
23         );  
24     for (auto& result: results)  
25         std::cout << result.get() << std::endl;  
26 }
```


Suggested Readings

- Chapter 4 of the «Parallel Programming Concept and Practice» book
- C++ reference <https://en.cppreference.com/w/cpp>
- «C++ High Performance» second edition by B. Andrist, V. Sehr, Packt
 - <https://www.packtpub.com/en-us/product/c-high-performance-9781787120952>
 - Git repo: <https://github.com/PacktPublishing/Cpp-High-Performance-Second-Edition>
 - Chapter 2 of this book provided as additional material
- «C++ Concurrency in Action» second edition by Anthony Williams
 - <https://www.manning.com/books/c-plus-plus-concurrency-in-action-second-edition>